

EE 5239 Nonlinear Optimization

Lecture 14a: The Alternating Direction Method of Multipliers (I)

Mingyi Hong, Jiaxiang Li
University of Minnesota

Outline

- 1 Review
- 2 Applications
- 3 The Algorithm

Last Time

- Last few lectures introduced ways to impose structure in the solution
- Extremely useful when problem size becomes large –
low-dimensional representation of high-dimensional data
- Adding penalty to the loss function:
 - ① Sparsity – $\|\cdot\|_0$ norm
 - ② Group sparsity – $\|\{\|\mathbf{x}_g\|_2\}_{g \in G}\|_0$ norm
 - ③ Low rank – $\text{Rank}(\mathbf{X})$
- These norms are difficult to deal with (even discontinuous)
- Use their convex surrogates
 - ① Sparsity – $\|\cdot\|_1$ norm
 - ② Group sparsity – $\|\{\|\mathbf{x}_g\|_2\}_{g \in G}\|_1 = \sum_{g \in G} \|\mathbf{x}_g\|_2$ norm
 - ③ Low rank – $\|\mathbf{X}\|_*$

Last Time

- These surrogates are nonsmooth but convex functions
- They share an important property: computing the so-called proximity operator is easy

$$\text{prox}_R^\gamma(\mathbf{y}) = \arg \min_{\mathbf{x}} \frac{1}{2} \|\mathbf{x} - \mathbf{y}\|^2 + \gamma R(\mathbf{x})$$

- Then we can use first-order method to solve the resulting problems
 - 1 Perform a simple gradient step
 - 2 Compute the proximity operator on the resulting objects

see <https://www.stat.cmu.edu/~ryantibs/convexopt/lectures/prox-grad.pdf>

The Anomaly Detection Problem?

$$\begin{array}{ll}\min & \|\mathbf{L}\|_* + \gamma_1 \|\mathbf{S}\|_1 + \gamma_2 \|\mathbf{Z}\|_F^2 \\ \text{subject to} & \mathbf{L} + \mathbf{S} + \mathbf{Z} = \mathbf{M}\end{array}$$

$$\min \quad \|\mathbf{L}\|_* + \gamma_1 \|\mathbf{S}\|_1 + \gamma_2 \|\mathbf{M} - \mathbf{L} - \mathbf{S}\|_F^2$$

❶ How to solve it?

This Time

- Introduce a family of algorithm called Alternating Direction Method of Multipliers (ADMM)
- Popular algorithm for large-scale machine learning, distributed computation, etc.
- Lots of applications
- Our discussion will be based on the following review paper:
“Distributed Optimization and Statistical Learning via the Alternating Direction Method of Multipliers”, S. Boyd, N. Parikh, E. Chu, B. Peleato, J Eckstein, *Foundation and Trends in Machine Learning*, 2011

Outline

- 1 Review
- 2 Applications
- 3 The Algorithm

Motivation

- **An Ancient Strategy**
- “Divide et impera” – Julius Caesar (100- 44 BC)
- “Divide and Rule” – Sun Tzu, ART OF WAR (535-470 BC)
- In mathematical terms: Split and Alternate

The Problem to be Solved

- The ADMM algorithm solves the following convex program

$$\begin{array}{ll}\min & f(\mathbf{x}) + g(\mathbf{z}) \\ \text{s.t.} & \mathbf{Ax} + \mathbf{Bz} = \mathbf{c} \\ & \mathbf{x} \in X, \quad \mathbf{z} \in Z\end{array}$$

- $\mathbf{x} \in \mathbb{R}^n$ and $\mathbf{z} \in \mathbb{R}^m$ are the variables
- f, g are two convex functions, possibly nonsmooth
- $\mathbf{A}, \mathbf{B}$ are two known matrices, $\mathbf{c}$ is a known vector
- Note:** Two blocks of variables; separable in the objective, coupled by a linear equation
- Main Benefit:** Capable of dealing with $\mathbf{x}$ and $\mathbf{z}$ separately (in a BCD manner)

Application 1: Constrained ℓ_1 problem

$$\begin{array}{ll}\min & \|x\|_1 \\ \text{s.t.} & \|Ax - b\|^2 \leq \delta\end{array}$$

- Similar as the LASSO, but with explicit constraints on the noise magnitude
- Formulate into the ADMM form?
- Introduce a new variable **z** (**split!**), arrive at an **equivalent formulation**

$$\begin{array}{ll}\min & \|x\|_1 \\ \text{s.t.} & \|z\|^2 \leq \delta \\ & z = Ax - b\end{array}$$

- Mapping back to the original problem, $f(x) = \|x\|_1$, $g(z) = 0$
- $B = -I$, $Z := \{z \mid \|z\|^2 \leq \delta\}$, $X = \mathbb{R}^n$

Application 2: Multiple Regularizations

- Consider the following **sparse group LASSO** problem
- The problem is formulated as

$$\min \frac{1}{2} \|\mathbf{Ax} - \mathbf{b}\|^2 + \underbrace{\sum_{g=1}^G \lambda_g \|\mathbf{x}_g\|_2}_{\ell_1/\ell_2 \text{ norm}} + \underbrace{\lambda \|\mathbf{x}\|_1}_{\ell_1 \text{ norm}}$$

- Select a few groups, and select a few variables within groups
- How to deal with **two** different nonsmooth penalizations simultaneously?

Application 2: Multiple Regularizations

- Same trick, **split and alternate**
- Introduce two new variables **$\mathbf{z}_1, \mathbf{z}_2$**
- Perform the following reformulation

$$\min \quad \underbrace{\sum_{g=1}^G \lambda_g \|\mathbf{x}_g\|_2}_{f(\mathbf{x})} + \underbrace{\frac{1}{2} \|\mathbf{A}\mathbf{z}_2 - \mathbf{b}\|^2 + \lambda \|\mathbf{z}_1\|_1}_{g(\mathbf{z})}$$

subject to **$\mathbf{z}_1 = \mathbf{x}, \quad \mathbf{z}_2 = \mathbf{x}$**

- Reformulated in an ADMM form; deal with two norms separately?

Application 3: Intersection of Two Sets

- Suppose we would like to solve the following problem

$$\begin{array}{ll} \text{FIND } \mathbf{x} \\ \text{subject to } & \|\mathbf{x}\|_1 \leq \delta, \quad \|\mathbf{Ax} - \mathbf{b}\|^2 \leq \epsilon \end{array} \quad (0.1)$$

- Finding the intersection of two convex sets
- We can introduce two variables $\mathbf{z}_1, \mathbf{z}_2$

$$\begin{array}{ll} \text{FIND } \mathbf{x} \\ \text{subject to } & \|\mathbf{z}_1\|_1 \leq \delta, \quad \|\mathbf{z}_2\|^2 \leq \epsilon \\ & \mathbf{z}_1 = \mathbf{x}, \quad \mathbf{z}_2 = \mathbf{Ax} - \mathbf{b} \end{array}$$

- First block $\mathbf{x}$, second block $(\mathbf{z}_1, \mathbf{z}_2)$; Fix $\mathbf{x}$, $\{\mathbf{z}_i\}$'s are independent!
- Dealing with each constraint separately

Application 4: Consensus Problem

- Consider the simple least square problem

$$\min \quad \|\mathbf{A}\mathbf{x} - \mathbf{b}\|^2 \quad (0.2)$$

- Suppose $\mathbf{A}$ is divided into N groups of rows $\mathbf{A}_1, \dots, \mathbf{A}_N$
- There are N computing nodes, each obtaining a single piece of data $\mathbf{A}_n$
- The problem can be re-written as

$$\min \quad \sum_{i=1}^N \|\mathbf{A}_i \mathbf{x} - \mathbf{b}_i\|^2 \quad (0.3)$$



Application 4: Consensus Problem

- Distributed computation?
- Let us introduce N variables $\mathbf{z}_1, \dots, \mathbf{z}_n$
- Consider the following reformulation

$$\begin{aligned} \min \quad & \sum_{i=1}^N \|\mathbf{A}_i \mathbf{z}_i - \mathbf{b}_i\|^2 \\ \text{subject to} \quad & \mathbf{z}_i = \mathbf{x}, \forall i \end{aligned}$$

- Two-blocks: $\mathbf{x}, (\mathbf{z}_1, \dots, \mathbf{z}_n)$
- Each $\mathbf{z}_i$ can be handled by the i -th local agent (together with $\mathbf{A}_i$)!
- Fix $\mathbf{x}$, $\{\mathbf{z}_i\}$'s are **independent**!

Application 4: Consensus Problem

- More generally, consider the following problem

$$\min \quad \sum_i f_i(\mathbf{x}) + R(\mathbf{x}), \text{ subject to } \mathbf{x} \in X \quad (0.4)$$

- We can perform the same procedure as before

$$\begin{aligned} \min \quad & \sum_i f_i(\mathbf{z}_i) + R(\mathbf{x}) \\ \text{subject to } & \mathbf{x} \in X, \mathbf{z}_i = \mathbf{x}, \forall i \end{aligned}$$

- Each agent handles a single function f_i
- Required to reach a **global consensus**
- Not necessarily to have the **“star network”**

Consensus SVM (from Boyd)

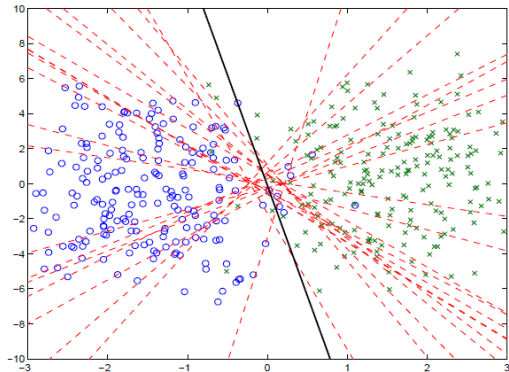
- Data $(\mathbf{a}_i, b_i), i = 1, \dots, N, \mathbf{a}_i \in \mathbb{R}^K, y_i \in \{-1, +1\}$
- Linear SVM

$$\min_{\mathbf{w}} \frac{1}{N} \sum_{i=1}^N \max \left\{ 0, 1 - b_i \mathbf{x}^T \mathbf{a}_i \right\} + \frac{\lambda}{2} \|\mathbf{x}\|_2^2 \quad (0.5)$$

- Example: $K = 2, N = 400$, splits into 20 groups

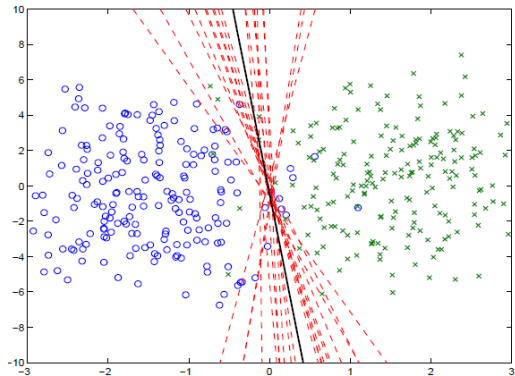
Consensus SVM (from Boyd)

Iteration 1



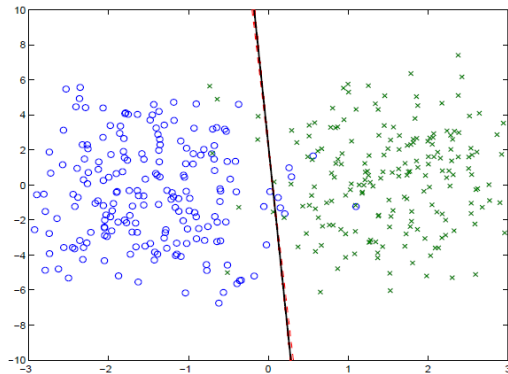
Consensus SVM (from Boyd)

Iteration 5



Consensus SVM (from Boyd)

Iteration 40



Application 5: Sharing Problem

- Consider the following sharing problem

$$\min \sum_{i=1}^N f_i(\mathbf{x}_i) + g\left(\sum_{i=1}^N \mathbf{x}_i\right) \quad (0.6)$$

- f_i local cost function; g shared objective
- Important class of problem, applications for example in economics
- Agents have selfish interests, but their behaviors are coupled together by some sort of “mutual interactions”

Application 5: Sharing Problem

- **First Approach** Introducing a new variable $\mathbf{z}$

$$\min \sum_{i=1}^N f_i(\mathbf{x}_i) + g(\mathbf{z})$$
$$\mathbf{z} = \sum_{i=1}^N \mathbf{x}_i$$

- **Second Approach** Introducing a set of new variable $\{\mathbf{z}_i\}$

$$\min \sum_{i=1}^N f_i(\mathbf{x}_i) + g\left(\sum_{i=1}^N \mathbf{z}_i\right)$$
$$\mathbf{z}_i = \mathbf{x}_i$$

Outline

- 1 Review
- 2 Applications
- 3 The Algorithm

Review: Dual Problem for Constrained Optimization

- Primal problem (with equality constraints)

$$(P) : \min f(\mathbf{x}), \quad \text{subject to } \mathbf{Ax} = \mathbf{b}, \mathbf{x} \in X \quad (0.1)$$

$f(\cdot)$ is a convex function

- Consider the Lagrangian, $\mathbf{y}$ is the dual variable (multiplier)

$$L(\mathbf{x}, \mathbf{y}) = f(\mathbf{x}) + \langle \mathbf{y}, \mathbf{Ax} - \mathbf{b} \rangle$$

- Dual function

$$d(\mathbf{y}) = \min_{\mathbf{x} \in X} L(\mathbf{x}, \mathbf{y}) \leq \min_{\mathbf{Ax} = \mathbf{b}, \mathbf{x} \in X} f(\mathbf{x})$$

- Dual problem (unconstrained)

$$(D) : \max_{\mathbf{y}} d(\mathbf{y}) = \max_{\mathbf{y}} \min_{\mathbf{x} \in X} L(\mathbf{x}, \mathbf{y}) \leq \min_{\mathbf{Ax} = \mathbf{b}, \mathbf{x} \in X} f(\mathbf{x})$$

- f convex, X convex set, strong duality, $P = D$ (optimal objective equals)

Dual Ascent

- Dual objective is concave

$$(D) : \max_{\mathbf{y}} d(\mathbf{y}) = \max_{\mathbf{y}} \min_{\mathbf{x}} L(\mathbf{x}, \mathbf{y}) = \max_{\mathbf{y}} \min_{\mathbf{x}} f(\mathbf{x}) + \langle \mathbf{y}, \mathbf{Ax} - \mathbf{b} \rangle$$

- Gradient Ascent for the dual problem

$$\mathbf{y}^{r+1} = \mathbf{y}^r + \alpha \nabla d(\mathbf{y}^r)$$

- Recall our dual SVM problem and algorithm?

Dual Ascent (cont.)

- Let $\tilde{\mathbf{x}} = \arg \min_{\mathbf{x} \in X} L(\mathbf{x}, \mathbf{y}^r)$, we know (can be verified)

$$\nabla d(\mathbf{y}^r) = \mathbf{A}\tilde{\mathbf{x}} - \mathbf{b}$$

- Dual Ascent Algorithm** (intuition?)

$$\mathbf{x}^{r+1} = \arg \min_{\mathbf{x} \in X} L(\mathbf{x}, \mathbf{y}^r)$$

$$\mathbf{y}^{r+1} = \mathbf{y}^r + \alpha (\mathbf{A}\mathbf{x}^{r+1} - \mathbf{b})$$

Dual Decomposition

- If f is **separable among variables**

$$(P) : \min_{\sum_{i=1}^N \mathbf{A}_i \mathbf{x}_i = \mathbf{b}} f(\mathbf{x}) = \sum_{i=1}^N f_i(\mathbf{x}_i) \quad (0.2)$$

- Consider the Lagrangian

$$L(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^N L_i(\mathbf{x}_i, \mathbf{y}) = \sum_{i=1}^N f_i(\mathbf{x}_i) + \langle \mathbf{y}, \mathbf{A}_i \mathbf{x}_i - \mathbf{b} \rangle \quad (0.3)$$

- Dual ascent algorithm (a.k.a. dual decomposition)

$$\begin{aligned} \mathbf{x}_i^{r+1} &= \arg \min_{\mathbf{x}_i \in X_i} L_i(\mathbf{x}_i, \mathbf{y}^r), \quad \forall i \\ \mathbf{y}^{r+1} &= \mathbf{y}^r + \alpha \left(\sum_{i=1}^N \mathbf{A}_i \mathbf{x}_i^{r+1} - \mathbf{b} \right) \end{aligned}$$

- Large-scale problem: Split variables, update $\mathbf{x}_i$ in **parallel**

Convergence of Dual Decomposition

- Assumption for convergence
 - ① Dual function $d(\mathbf{y})$ has **Lipschitz continuous gradient** (with constant L)
 - ② The stepsize $\alpha \leq \frac{1}{L}$
- (1) requires that problem (P) is **strongly convex** with constant at least $\frac{1}{L}$
- Consider again the dual ascent algorithm we tried for the SVM problem

Method of Multipliers

- Primal problem (with equality constraints)

$$(P) : \min f(\mathbf{x}), \quad \text{subject to } \mathbf{Ax} = \mathbf{b}, \mathbf{x} \in X \quad (0.4)$$

$f(\cdot)$ is a convex function

- Consider the **Augmented Lagrangian**, $\mathbf{y}$ is the dual variable (multiplier)

$$L(\mathbf{x}, \mathbf{y}) = f(\mathbf{x}) + \langle \mathbf{y}, \mathbf{Ax} - \mathbf{b} \rangle + \frac{\rho}{2} \|\mathbf{Ax} - \mathbf{b}\|^2$$

- $\rho > 0$ is penalty parameter
- Quadratic penalty term penalizes constraint violation
- **Method of Multipliers** (intuition?)

$$\mathbf{x}^{r+1} = \arg \min_{\mathbf{x} \in X} L_{\rho}(\mathbf{x}, \mathbf{y}^r)$$

$$\mathbf{y}^{r+1} = \mathbf{y}^r + \rho (\mathbf{Ax}^{r+1} - \mathbf{b})$$

Method of Multipliers

- **Good**
 - ① Weak Assumptions: f only needs to be convex
 - ② Dual function has Lipschitz continuous gradient
- **Bad**
 - ① Objective not separable anymore
 - ② Dual decomposition no longer works
- ADMM combines
 - ① Method of multipliers: Use **augmented Lagrangian**
 - ② Dual decomposition: Can do variable splitting